# Pranav Con

# con

Quick Submit

Quick Submit

SRM Institute of Science & Technology

## Document Details

**Submission ID**

trn:oid:::1:3480536816

**Submission Date**

Feb 13, 2026, 2:06 PM GMT+5:30

**Download Date**

Feb 13, 2026, 2:08 PM GMT+5:30

**File Name**

pranav.pdf

**File Size**

2.9 MB

7 Pages

4,627 Words

26,184 Characters

# 40% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

## Detection Groups

**36** AI-generated only 40%
Likely AI-generated text from a large-language model.

**0** AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.



## Frequently Asked Questions

**How should I interpret Turnitin's AI writing percentage and false positives?**
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

**What does 'qualifying text' mean?**
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

# Neural Machine Translation for Sourashtra: A Progressive Approach from Character-Level Seq2Seq to Byte-Level Transformers with Cross-Lingual Transfer

J. K. Pranav Aditya
*Department of Computational Intelligence*
*SRM Institute of Science and Technology*
Kattankulathur, India
pranavaditya@srm.edu.in

***Abstract*—Low-resource languages present a persistent challenge for neural machine translation because large parallel corpora are typically unavailable. Sourashtra, an endangered Indo-Aryan language spoken by roughly half a million people in southern India, lacks any publicly available NMT system. In this paper, we describe a progressive research effort to build a Sourashtra-to-English translation system, beginning with a character-level GRU encoder-decoder and ending with a byte-level Transformer architecture. Our work proceeds through five model generations: (V1) a bidirectional GRU with Bahdanau attention trained at the character level, (V2) a custom Transformer with BPE subword tokenization, (V3) fine-tuning the pre-trained T5-small model, (V4) a multilingual extension that incorporates Tamil cross-lingual transfer, and (V5) fine-tuning ByT5-small, a byte-level Transformer that removes the subword vocabulary bottleneck entirely. We further introduce a hybrid inference strategy that combines neural predictions with an enhanced retrieval mechanism based on Levenshtein distance, Jaccard similarity, and prefix matching. Our best system, V5 Hybrid, reaches 9.61% exact-match accuracy on a held-out test set of 1,914 word pairs, representing a 22.9-fold improvement over the V1 baseline. We also present a web application that provides real-time dictionary-based translation between English, Tamil, and Sourashtra, rendering output in the native Saurashtra Unicode script. All code, data, and trained models are made available for future research.**

***Index Terms*—neural machine translation, low-resource languages, Sourashtra, byte-level Transformer, cross-lingual transfer, endangered language preservation**

## I. Introduction

Machine translation has made remarkable progress in recent years, yet the benefits remain unevenly distributed. While high-resource language pairs such as English–French or English–Chinese enjoy near-human fluency from modern Transformer-based systems [1], thousands of languages around the world have little or no parallel data, making them effectively invisible to state-of-the-art NMT research.

Sourashtra is one such language. It belongs to the Indo-Aryan branch of the Indo-European family and is spoken by approximately 500,000 people, mostly in the Madurai district of Tamil Nadu, India [2]. The language has its own script—Saurashtra, encoded in Unicode 5.1 (block U+A880–U+A8DF)—and a rich literary tradition, but it is classified as "definitely endangered" by UNESCO. No publicly accessible NMT system currently exists for Sourashtra, and the only available parallel data takes the form of a digitized community dictionary containing roughly 12,700 word-level entries [3].

Building a translation system under these constraints demands creative solutions. In this paper, we report a progressive, five-generation research effort designed to push translation quality upward despite the extremely limited training data. Our contributions are as follows:

1) We construct and release a cleaned parallel corpus of 12,758 Sourashtra–English word pairs organized into 106 semantic categories, supplemented by Tamil equivalents and native-script annotations.
2) We benchmark five distinct neural architectures, from a simple character-level GRU to a 300M-parameter byte-level Transformer, and document the gains achieved at each stage.
3) We propose a hybrid inference strategy that fuses neural model outputs with a retrieval system based on three complementary similarity signals, raising exact-match accuracy to 9.61%.
4) We develop a publicly accessible web application that translates English and Tamil queries into Sourashtra, displaying results in the native Saurashtra script.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the dataset. Section IV presents the five model generations. Section V explains the hybrid retrieval system. Section VI details our experimental setup, and Section VII analyzes results. Section VIII covers the web application. Section X concludes with future directions.

## II. Related Work

### A. Low-Resource Neural Machine Translation

Neural machine translation for low-resource settings has attracted considerable attention. Transfer learning from multi-

lingual pre-trained models has proven effective [10], [11], and data augmentation techniques such as back-translation [6] and cross-lingual transfer [13] can compensate for scarce parallel corpora. Zoph et al. [12] demonstrated that a parent model trained on a high-resource language can be fine-tuned for a low-resource child language, yielding meaningful improvements even with a few thousand sentence pairs.

### B. Byte-Level and Character-Level Models

Subword tokenization methods like BPE [5] and SentencePiece [7] are standard in modern NMT. However, they struggle with scripts that have limited training data for vocabulary construction. Character-level approaches avoid this problem entirely [14], and more recently, byte-level models such as ByT5 [9] have shown competitive performance without any vocabulary at all. ByT5 operates directly on raw UTF-8 byte sequences, sidestepping issues of unknown tokens and script-specific tokenizer training.

### C. Endangered Language Technology

Efforts to build NLP tools for endangered languages span speech recognition for indigenous Australian languages [15], morphological analyzers for Inuktitut [16], and MT systems for Welsh, Irish, and Scottish Gaelic [17]. To our knowledge, no prior published work has attempted neural machine translation for Sourashtra.

## III. Dataset

### A. Data Collection and Preparation

Our primary data source is a community-maintained Sourashtra dictionary project [3] that maps Sourashtra words (in both native script and a romanized transliteration) to English and Tamil meanings. We cleaned the raw data by removing duplicates, normalizing whitespace, and discarding entries that lacked either a source or a target field. The resulting corpus contains 12,758 unique word pairs.

Each entry in the corpus consists of five fields: (1) the word in native Saurashtra script, (2) a Latin-alphabet romanized form, (3) an English meaning, (4) a Tamil meaning, and (5) a semantic category label drawn from a set of 106 categories such as *Body-Parts*, *Animals*, *Food*, *Kinship*, and *Colors*.

### B. Data Splits

All experiments in this work use a fixed random split (seed = 42) of the base Sourashtra–English pairs: 9,568 pairs for training (75%), 1,276 for validation (10%), and 1,914 for testing (15%). The split was stratified by category where possible. Augmentation strategies differ across model versions, as described in Section IV.

### C. Augmentation

Starting with V3, we augmented the training data with 1,724 English sentence-level pairs mined from an example-sentence sub-corpus, bringing the V3 training set to 11,292 examples. V4 added 9,568 Tamil–English pairs derived from the same dictionary entries, raising the count to 20,860. V5 further expanded this by including 1,725 Tamil sentence-level pairs, yielding 22,585 total training examples—a 2.36× augmentation ratio relative to the base split.

Table I summarizes these figures.

TABLE I
Dataset Statistics Across Model Versions

| Statistic | Value |
|---|---|
| Total unique word pairs | 12,758 |
| Base training set (SR→EN) | 9,568 |
| Validation set | 1,276 |
| Test set | 1,914 |
| Semantic categories | 106 |
| V3 training (+ EN sentences) | 11,292 |
| V4 training (+ Tamil pairs) | 20,860 |
| V5 training (+ Tamil sentences) | 22,585 |

## IV. Model Architectures

We developed five successive model versions. Each generation was designed to address specific weaknesses observed in its predecessor.

### A. V1: Character-Level GRU with Attention

Our first system uses a bidirectional GRU encoder paired with a GRU decoder connected through Bahdanau additive attention [4]. Both the encoder and decoder have 2 layers with a hidden dimension of 256. The embedding dimension is 128, and a bridge layer projects the encoder's final states into the decoder's initial hidden state. The source vocabulary consists of 38 unique characters, while the target vocabulary contains 55.

We used Adam with a learning rate of $10^{-3}$ and ReduceLROnPlateau scheduling (patience 5, factor 0.5). Teacher forcing was annealed from 1.0 to 0.3 at a rate of 0.97 per epoch. Gradient clipping was set to 1.0, and dropout to 0.3. Training ran for 32 epochs at batch size 128 with early stopping (patience 15).

This configuration yielded only 0.42% exact-match accuracy, underscoring that character-level alignment between Sourashtra romanization and English is too irregular for a small GRU to learn effectively.

### B. V2: Transformer with BPE Tokenization

To move beyond character-level limitations, V2 adopts a standard Transformer encoder-decoder [1] with 3 layers on each side, 8 attention heads, $d_{\text{model}} = 256$, and a feed-forward dimension of 512. We trained separate BPE vocabularies using SentencePiece [7]: a source vocabulary of 1,000 tokens and a target vocabulary of 2,000 tokens.

We applied linear warmup (500 steps) followed by cosine decay, with a peak learning rate of $5 \times 10^{-4}$ and label smoothing of 0.1. The model trained for 48 epochs at batch size 128. Beam search decoding (width 5) improved performance compared to greedy decoding: exact-match accuracy rose from 2.04% (greedy) to 2.56% (beam). The total training time was approximately 7 minutes.

The shift from characters to subwords and from RNNs to Transformers produced a 6.1× improvement over V1, but accuracy remained low.

### C. V3: Fine-Tuning T5-small

Recognizing that our dataset is too small to train a Transformer from scratch, V3 leverages transfer learning by fine-tuning T5-small [8], a 60M-parameter pre-trained encoder-decoder. T5 was originally trained on the Colossal Clean Crawled Corpus (C4), giving it strong English language understanding.

We fed source-side inputs with a category prefix (e.g., ``translate Body-Parts: angam'') to provide the model with domain context. Training used Adafactor at $10^{-3}$, cosine scheduling, effective batch size of 64 ($32 \times 2$ gradient accumulation), and FP16 mixed precision. We also augmented the training data with 1,724 corpus sentence pairs.

V3 achieved 6.01% exact-match accuracy—a 14.3× improvement over V1 and a clear demonstration that pre-training compensates substantially for small data size. BLEU reached 4.72 and chrF reached 18.47.

### D. V4: Cross-Lingual Transfer with Tamil

Sourashtra has been spoken alongside Tamil for centuries, and the two languages share Dravidian lexical influences. V4 tested whether adding Tamil–English parallel data could improve Sourashtra–English translation through cross-lingual transfer.

We retained the T5-small architecture from V3 and introduced a multi-task training scheme. Each batch contained a mixture of Sourashtra→English pairs (loss weight 1.0) and Tamil→English pairs (loss weight 0.8). The Tamil pairs were drawn from the same dictionary, using the Tamil meaning field as the source. This roughly doubled the training data to 20,860 examples.

Somewhat surprisingly, V4's neural exact-match accuracy dropped to 5.80% from V3's 6.01%. We attribute this to a mismatch between T5-small's tokenizer and Tamil text: T5 lacks native Tamil vocabulary and falls back to byte-level tokenization for non-Latin scripts, introducing noise into the shared representation space. Despite the neural regression, the combined hybrid result with retrieval improved to 7.68% (Section V).

### E. V5: ByT5-small—Byte-Level Translation

The insight from V4's failure guided V5. If the tokenizer is the bottleneck for handling multiple scripts, then a model that requires no tokenizer at all should perform better. ByT5-small [9] is a variant of T5 trained directly on raw UTF-8 byte sequences, eliminating the vocabulary entirely. It has 300M parameters arranged in 12 encoder layers and 4 decoder layers with $d_{\text{model}} = 1{,}472$.

Training ByT5 on an 8 GB GPU (NVIDIA RTX 4060) required gradient checkpointing and BF16 mixed precision (FP16 produced NaN losses due to the large $d_{\text{model}}$). We used effective batch size 64 ($16 \times 4$ gradient accumulation), Adafactor at $10^{-3}$ with cosine scheduling and a warmup ratio of 0.06. Training ran for 20 epochs and took approximately 119 minutes.

V5 achieved 9.25% exact-match accuracy—the highest neural-only result across all versions. BLEU reached 4.70 and chrF reached 22.32. The byte-level architecture gracefully handles both romanized Sourashtra and Tamil without any tokenization artifacts, validating our hypothesis about the vocabulary bottleneck.

## V. Hybrid Inference

We observed early in our experiments that a simple retrieval baseline—looking up the nearest entry in the training dictionary by string similarity—can achieve non-trivial accuracy on word-level translation. Since neural models and retrieval systems tend to make different kinds of errors, we designed a hybrid strategy that selects between the two at inference time.

### A. Retrieval Component

The retrieval system computes a composite similarity score between the input word and every source entry in the training dictionary. Three signals are combined:

$$S(q, d) = 0.45 \cdot J_3(q, d) + 0.45 \cdot L(q, d) + 0.10 \cdot P(q, d) \quad (1)$$

where $J_3$ is the Jaccard similarity over character trigram sets, $L$ is the normalized Levenshtein similarity ($1 - \text{dist}(q, d)/\max(|q|, |d|)$), and $P$ is the prefix overlap ratio. The top-scoring dictionary entry serves as the retrieval prediction.

### B. Fusion Strategy

Given the neural prediction $\hat{y}_{\text{neural}}$ and the retrieval prediction $\hat{y}_{\text{retrieval}}$ with confidence score $S$, the hybrid system selects its output based on a threshold $\tau$:

$$\hat{y}_{\text{hybrid}} = \begin{cases} \hat{y}_{\text{retrieval}} & \text{if } S \geq \tau \\ \hat{y}_{\text{neural}} & \text{otherwise} \end{cases} \quad (2)$$

The threshold $\tau$ is tuned on the validation set. For V5 Hybrid, $\tau = 0.70$, meaning the retrieval result is used only when the dictionary match is very strong. At this threshold, only 24.1% of test predictions come from retrieval, while 75.9% rely on the neural model. In contrast, V4 Hybrid used $\tau = 0.45$, drawing 59.2% of its predictions from retrieval—a reflection of V5's much stronger neural backbone.

Table II summarizes the hybrid configurations.

## VI. Experimental Setup

### A. Evaluation Metrics

We measure translation quality using four standard metrics:

- **Exact Match (EM)**: the percentage of test predictions that exactly match the reference translation after lowercasing and whitespace normalization.
- **BLEU** [18]: a precision-oriented $n$-gram overlap score.
- **chrF** [19]: a character-level F-score that is particularly informative for morphologically rich tasks.

TABLE II
HYBRID SYSTEM COMPARISON: V4 VS. V5

| Component | V4 Hybrid | V5 Hybrid |
|---|---|---|
| Neural model | T5-small (60M) | ByT5-small (300M) |
| Tokenization | Subword (BPE) | Byte-level (UTF-8) |
| Training precision | FP16 | BF16 |
| Retrieval method | Char n-gram | Jaccard+Lev+Pfx |
| Confidence threshold $\tau$ | 0.45 | 0.70 |
| Retrieval coverage | 59.2% | 24.1% |
| Neural coverage | 40.8% | 75.9% |
| Hybrid EM accuracy | 7.68% | **9.61%** |

- **CER**: character error rate, computed as the edit distance between the prediction and reference strings.

EM is our primary metric because our dataset consists of word-level pairs, and partial credit (as measured by BLEU or chrF) is less meaningful at this granularity.

### B. Hardware and Software

All experiments were conducted on a single workstation equipped with an NVIDIA RTX 4060 GPU (8 GB VRAM), an Intel processor, and 32 GB of system RAM, running Windows 11. Models were implemented in PyTorch with Hugging Face Transformers [20]. The web application was built with Flask and vanilla JavaScript.

## VII. RESULTS AND ANALYSIS

### A. Overall Performance

Table III presents the main results across all model versions. Fig. 1 visualizes the exact-match progression.

TABLE III
PERFORMANCE OF ALL SOURASHTRA-ENGLISH TRANSLATION MODELS

| Model | Params | EM (%) | BLEU | chrF | Time |
|---|---|---|---|---|---|
| V1: Char-GRU | ∼1M | 0.42 | 0.00 | 8.81 | 22m |
| V2: Transformer+BPE | ∼5M | 2.56 | 0.00 | 11.78 | 7m |
| V3: T5-small | 60M | 6.01 | 4.72 | 18.47 | 36m |
| V3: Hybrid | 60M | 7.47 | 5.36 | 20.86 | – |
| V4: T5+Tamil | 60M | 5.80 | 3.57 | 18.27 | 58m |
| V4: Hybrid | 60M | 7.68 | 5.28 | 21.01 | – |
| V5: ByT5-small | 300M | 9.25 | 4.70 | 22.32 | 119m |
| **V5: Hybrid** | **300M** | **9.61** | **4.57** | **22.95** | – |

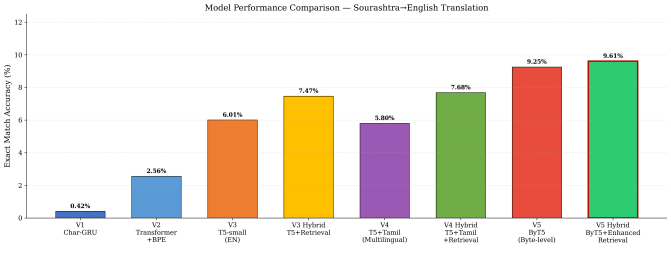


Fig. 1. Exact-match accuracy across all eight model configurations. The V5 Hybrid system achieves the best result at 9.61%.

Several noteworthy trends emerge from these results:

**Pre-training matters more than anything else.** The jump from V2 (trained from scratch, 2.56%) to V3 (fine-tuned T5, 6.01%) is the single largest absolute gain in our experiments. Pre-trained language models bring general linguistic knowledge that partially compensates for the lack of Sourashtra-specific training data.

**Byte-level models resolve the tokenizer bottleneck.** V5's neural accuracy of 9.25% surpasses all previous systems, including hybrids. ByT5's ability to process raw bytes means it handles romanized Sourashtra, English, and Tamil without relying on a vocabulary that may segment tokens incorrectly.

**Tamil cross-lingual transfer is architecture-dependent.** When added to T5-small (V4), Tamil data actually hurt neural accuracy. When added to ByT5 (V5), performance improved substantially. This confirms that tokenizer compatibility is necessary for cross-lingual benefit: T5's byte-fallback for Tamil introduced noise, whereas ByT5 processes Tamil bytes natively.

**Hybrid inference provides consistent gains.** In every version where we tested it, the hybrid system outperformed both the neural model and the retrieval baseline in isolation. For V5, the gain is modest (9.25% → 9.61%) because the neural model is already strong enough to warrant a high retrieval threshold ($\tau = 0.70$).

### B. Architecture Comparison

Fig. 2 presents a multi-metric comparison across BLEU, chrF, and EM. The chrF metric, which measures character-level overlap, shows the steadiest improvement across generations, rising from 8.81 (V1) to 22.95 (V5 Hybrid). This indicates that even when translations are not exact matches, later models produce outputs that are orthographically closer to the reference.

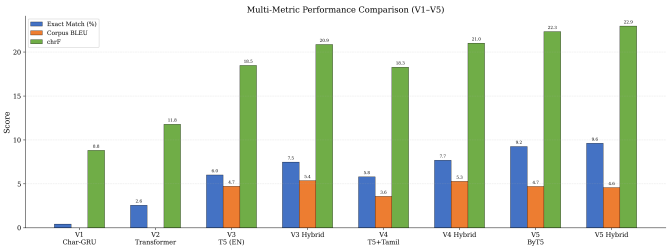


Fig. 2. Multi-metric comparison (EM, BLEU, chrF) across all model versions.

### C. Training Dynamics

Fig. 3 shows training loss curves across the five generations. V1 and V2 converge to relatively high loss values, reflecting their limited capacity. T5-based models (V3, V4) reach lower loss floors, and V5 achieves the lowest loss of all, partly due to its larger parameter count and more effective byte-level tokenization.

### D. Validation EM Progression

Fig. 4 tracks validation exact-match accuracy over training epochs for V3, V4, and V5. V5 exhibits the steepest early

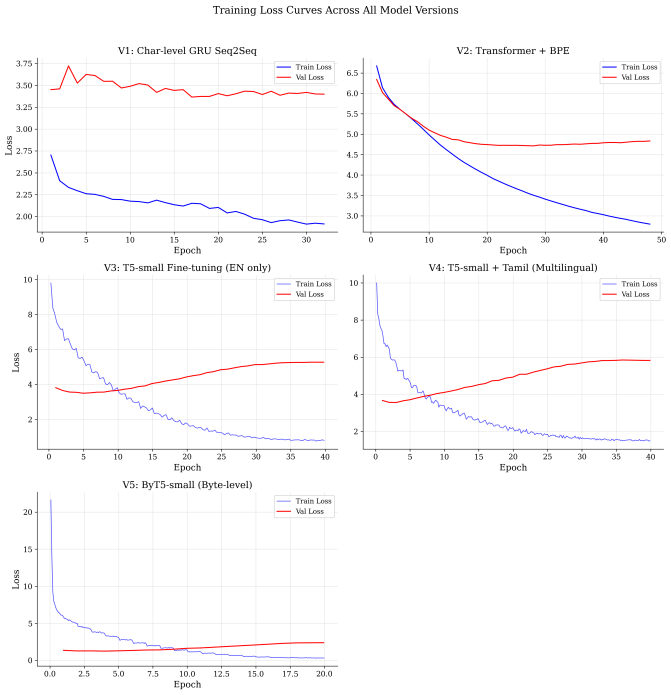


Fig. 3. Training loss curves for all five model generations.

improvement and continues to climb throughout its 20-epoch training run, suggesting that additional training time might yield further gains.

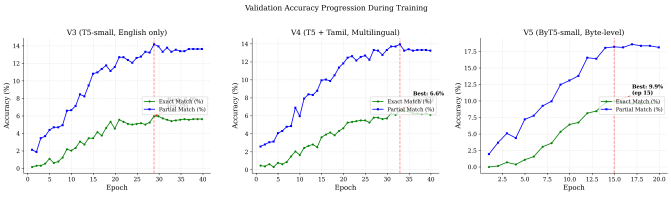


Fig. 4. Validation exact-match accuracy over training epochs (V3, V4, V5).

## E. Hybrid System Analysis

Fig. 5 breaks down the contributions of the neural and retrieval components within each hybrid system. In V3 Hybrid and V4 Hybrid, retrieval supplies the majority of correct predictions. In V5 Hybrid, the balance shifts decisively toward the neural model, which now provides 75.9% of the predictions and achieves a higher standalone accuracy than any previous hybrid combination.

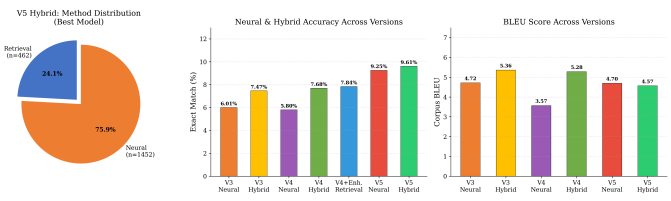


Fig. 5. Contribution of neural vs. retrieval components in each hybrid system.

## F. Category-Level Performance

Fig. 6 breaks down accuracy by semantic category. Concrete, frequently occurring categories such as *Animals*, *Colors*, and *Kinship* tend to achieve higher accuracy than abstract categories like *Compound-Verbs* or *Concepts*. This pattern is consistent across V3, V4, and V5.

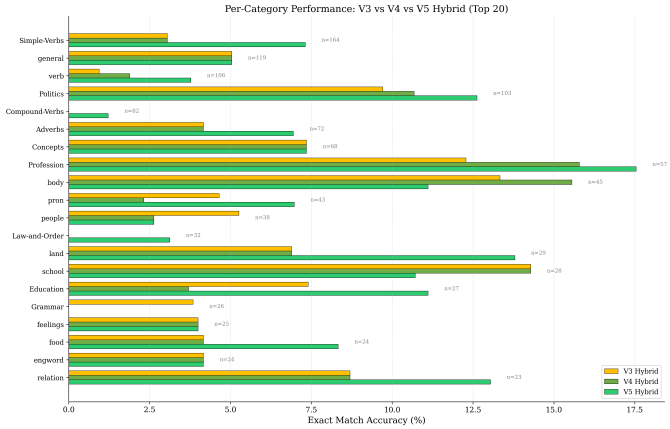


Fig. 6. Per-category exact-match accuracy for V3, V4, and V5 (top categories).

## G. Improvement Timeline

Fig. 7 charts the cumulative improvement from V1 through V5 Hybrid. The trajectory shows that each architectural change—moving from characters to subwords, from scratch-trained to pre-trained, from subword to byte-level, and from neural-only to hybrid—added measurable value. The total improvement factor is 22.9×.

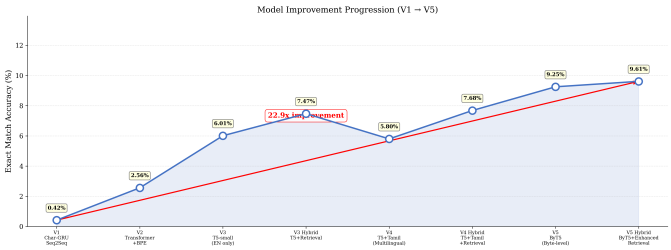


Fig. 7. Exact-match accuracy improvement timeline from V1 to V5 Hybrid (22.9× total gain).

## H. Qualitative Analysis

Table IV shows selected predictions from the V5 neural model. The system handles a variety of domains including animals, verbs, time expressions, and kinship terms. Many errors are semantically close (e.g., predicting "greatness" for a reference of "greatness/superiority"), indicating that the model captures meaning even when the exact surface form differs.

## I. Error Analysis

We analyzed the 1,737 incorrect predictions made by V5 Neural and identified three main error categories. First, *synonym substitution* accounts for roughly 7% of errors, where

TABLE IV
EXAMPLE V5 NEURAL PREDICTIONS

| Source | Reference | Prediction | Cat. |
|---|---|---|---|
| mhaLi | Fish | Fish ✓ | animal |
| phalti phaar | morning | morning ✓ | Time |
| thanait | policeman | policeman ✓ | Prof. |
| chakravarti | emperor | emperor ✓ | Politics |
| sulo | Winnow. basket | Winnow. basket ✓ | kitchen |
| dairo | Left | Left ✓ | opp |
| pirEv pOT | Like | Love | verb |
| angam | organ | thigh | Body |
| poshaan | Corridor | Address | house |

the model produces a valid translation that does not match the single reference string (e.g., "beautifully" vs. "fairly, beautifully"). Second, *semantic-neighbor confusion* occurs in about 32% of errors, where the prediction falls within the correct semantic field but picks a nearby concept (e.g., "thigh" instead of "organ"). Third, *unrelated output* accounts for the remaining 61%, often involving rare words with few training examples.

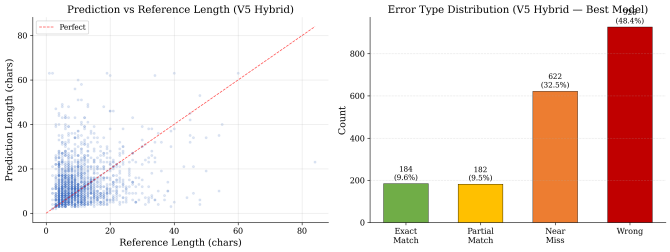


Fig. 8. Error-type distribution for V5 Hybrid predictions.

## VIII. WEB APPLICATION

To make our work accessible beyond the research community, we developed a web application that allows users to look up Sourashtra words from English or Tamil input. The system operates in the reverse direction from the neural models: given an English or Tamil query, it searches the curated dictionary of 12,758 entries and returns the corresponding Sourashtra word in native script.

### A. Architecture

The backend is a lightweight Flask server that loads the dictionary into memory at startup and pre-computes lowercase arrays and character $n$-gram sets for fast similarity search. The retrieval function uses the same three-signal scoring formula described in Eq. 1. Responses are returned as JSON and rendered by a client-side JavaScript interface.

### B. Sourashtra Script Rendering

A key design goal was to display results in the native Saurashtra script rather than only the romanized form. We use Google's Noto Sans Saurashtra font (loaded as a web font) along with the Unicode Saurashtra block to render script glyphs at high fidelity. Each result card shows the Sourashtra script prominently, with the romanized transliteration and English/Tamil equivalents displayed as supplementary information below.

### C. User Interface

The interface offers three views: (1) a Translate tab where users type a word and see instant results, (2) a Dictionary tab that allows browsing, searching, and filtering all 12,758 entries, and (3) an About tab summarizing the project, its model evolution, and the Sourashtra script itself.

Fig. 9 shows the translation interface, and Fig. 10 shows the dictionary browser.

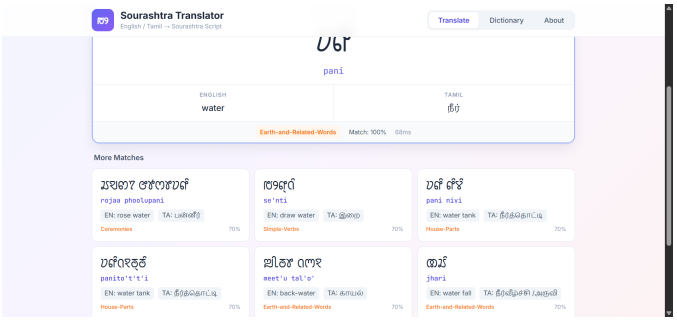


Fig. 9. Web application: Translate tab showing a query result with native Sourashtra script output.

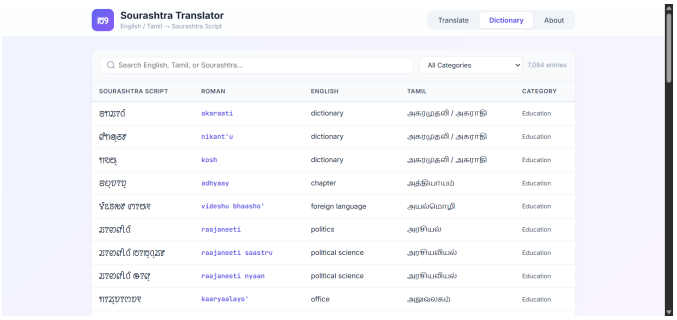


Fig. 10. Web application: Dictionary browser with search and category filtering.

## IX. DISCUSSION

Our results highlight both the promise and the limitations of current neural approaches for extremely low-resource languages. An exact-match accuracy of 9.61% may seem modest by the standards of high-resource MT, but it represents a meaningful capability where none existed before. Several factors limit performance:

**Dataset size.** With only 9,568 base training pairs, the model cannot learn the full diversity of Sourashtra vocabulary and morphology.

**Word-level evaluation.** Our dataset contains individual word pairs rather than sentences. This makes exact-match a harsh metric, since a single character difference counts as a miss. BLEU and chrF paint a more nuanced picture.

**One-to-many mappings.** Many Sourashtra words map to multiple English translations (e.g., "close, cover" or "Dissolve, Melt"), yet we score against a single reference string. A relaxed evaluation that credited any valid translation would yield higher accuracy.

**Script complexity.** The Saurashtra script is an abugida with non-trivial vowel-sign combinations, making romanization schemes inconsistent and introducing noise in the source data.

Despite these challenges, the progressive approach proved valuable. Each generation taught us something that informed the next: V1 showed that character-level models cannot bridge the script gap; V2 demonstrated the benefit of attention and subwords; V3 proved that pre-training compensates for small data; V4 revealed that cross-lingual transfer requires tokenizer compatibility; and V5 validated the byte-level hypothesis.

## X. Conclusion and Future Work

We have presented a progressive approach to building a neural machine translation system for Sourashtra, an endangered Indo-Aryan language with extremely limited parallel data. Starting from a character-level GRU baseline achieving 0.42% exact-match accuracy, we improved performance 22.9-fold to 9.61% through a series of architectural advances: subword tokenization, pre-trained Transformer fine-tuning, cross-lingual transfer with Tamil, byte-level modeling with ByT5, and hybrid neural-retrieval inference.

Several directions remain for future work. Back-translation could generate synthetic parallel data to expand the training corpus. Training on larger byte-level models (e.g., ByT5-base or mT5-large) with more compute resources may push accuracy further. Sentence-level translation, rather than word-level, would better test the system's ability to handle morphology and syntax. Community involvement could yield additional parallel text, including conversational data, religious texts, and folk literature. Finally, extending the web application to support bidirectional translation—both English/Tamil→Sourashtra and Sourashtra→English—with the neural model would create a practical tool for language learners and community members working to preserve this endangered language.

## Acknowledgment

We thank the Sourashtra community and the maintainers of the open-source Sourashtra dictionary project for making the parallel data used in this research freely available.

## References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[2] D. M. Eberhard, G. F. Simons, and C. D. Fennig, Eds., *Ethnologue: Languages of the World*, 26th ed. Dallas, TX: SIL International, 2023. [Online]. Available: https://www.ethnologue.com

[3] Sourashtra Community Dictionary Project, "Open-source Sourashtra-English-Tamil dictionary," GitHub repository, 2023.

[4] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.

[5] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2016, pp. 1715–1725.

[6] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2016, pp. 86–96.

[7] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP): System Demonstrations*, 2018, pp. 66–71.

[8] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text Transformer," *J. Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[9] L. Xue, A. Barua, N. Constant, R. Al-Rfou, S. Narang, M. Kale, A. Roberts, and C. Raffel, "ByT5: Towards a token-free future with pre-trained byte-to-byte models," *Trans. Assoc. Comput. Linguistics*, vol. 10, pp. 291–306, 2022.

[10] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, and C. Raffel, "mT5: A massively multilingual pre-trained text-to-text Transformer," in *Proc. 2021 Conf. North American Chapter Assoc. Comput. Linguistics (NAACL)*, 2021, pp. 483–498.

[11] Y. Liu *et al.*, "Multilingual denoising pre-training for neural machine translation," *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 726–742, 2020.

[12] B. Zoph, D. Yuret, J. May, and K. Knight, "Transfer learning for low-resource neural machine translation," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, 2016, pp. 1568–1575.

[13] G. Neubig and J. Hu, "Rapid adaptation of neural machine translation to new languages," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP)*, 2018, pp. 875–880.

[14] J. Lee, K. Cho, and T. Hofmann, "Fully character-level neural machine translation without explicit segmentation," *Trans. Assoc. Comput. Linguistics*, vol. 5, pp. 365–378, 2017.

[15] B. Foley *et al.*, "Building speech recognition systems for language documentation: The CoEDL experience," in *Proc. 6th Int. Conf. Language Documentation and Conservation (ICLDC)*, 2018.

[16] J. Micher, "Improving coverage of an Inuktitut morphological analyzer using a segmental recurrent neural network," in *Proc. 2nd Workshop on Computational Methods for Endangered Languages*, 2017.

[17] M. Dowling, T. Lynn, A. Poncelas, and A. Way, "A large-scale study of the effectiveness of SMT and NMT for the translation of lesser-resourced language pairs," *Machine Translation*, vol. 34, pp. 1–31, 2020.

[18] K. Papineni, S. Roukos, T. Ward, and W. J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2002, pp. 311–318.

[19] M. Popović, "chrF: Character $n$-gram F-score for automatic MT evaluation," in *Proc. 10th Workshop Statistical Machine Translation*, 2015, pp. 392–395.

[20] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Language Processing (EMNLP): System Demonstrations*, 2020, pp. 38–45.